EX NO:6A	K MEANS CLUSTERING
DATE:	

AIM:

To understand and implement K Means Clustering.

PROCEDURE :

- Procedure for K-Means Clustering
- Collect and preprocess the dataset by selecting the relevant input features.
- Choose the number of clusters (K) for grouping the data points.
- Initialize K cluster centroids randomly.
- Assign each data point to the nearest centroid based on distance.
- Recalculate the centroids by taking the mean of the data points in each cluster.
- Repeat the assignment and centroid update process until the clusters stabilize.
- Visualize the clusters to analyze the grouping of data points.

CODE :

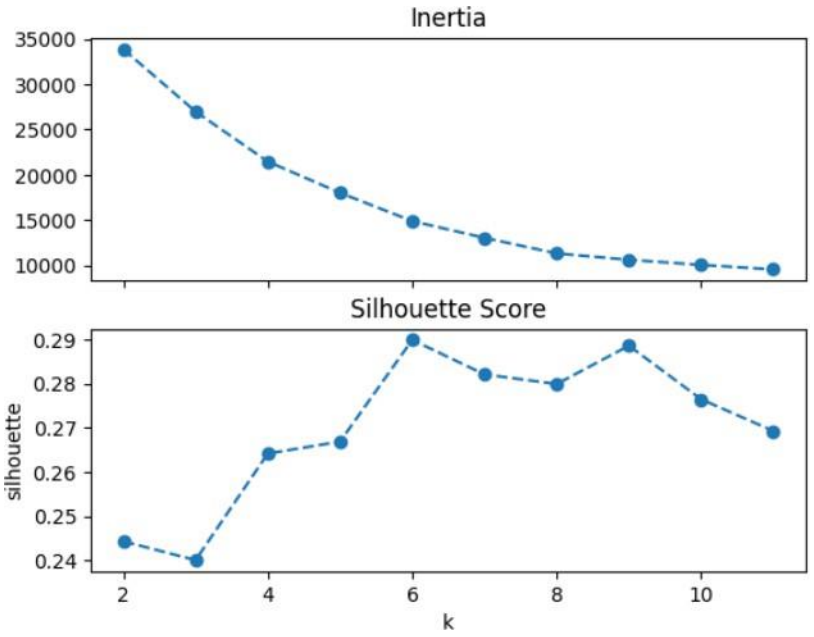
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
df=pd.read_csv(' /content/drive/MyDrive/717824I363/datasets/moive
recommendation.csv')
df.head()
df.describe()
```

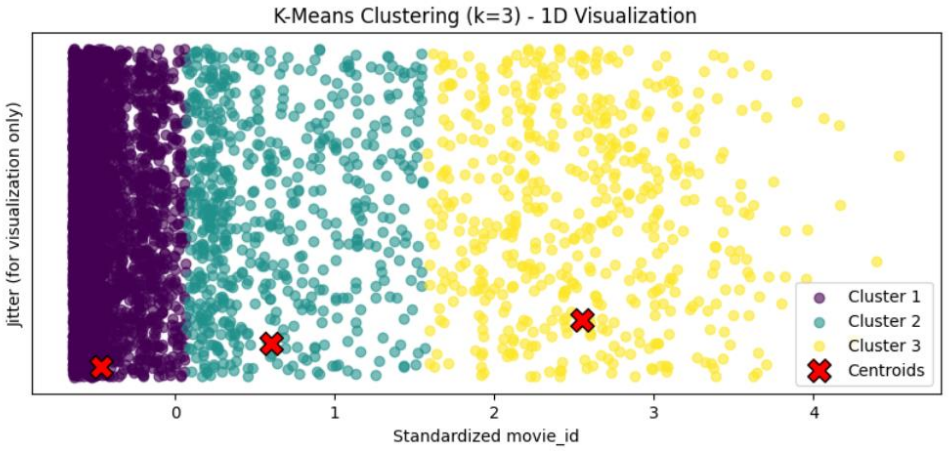
OUTPUT:

	movie_id	title	cast	crew	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	...	Unnamed: 1255	Unnamed: 1256
count	4812	4813	4813	4809	10	10	9	9	9	9	...	1	1
unique	4812	4808	4771	4782	10	7	9	7	9	9	...	1	1
top	25975	""gender"": 0	[]	[]	""name"": ""Ronn Wong""}	""gender"": 0	""department"": ""Costume & Make-Up""	""gender"": 0	""id"": 1463383	""job"": ""Set Dressing Artist""	...	""gender"": 0	""id"": 1645398 ""
freq	1	3	43	28	1	4	1	3	1	1	...	1	1

```
x=df
# Impute missing values with the median before scaling

scalar=StandardScaler()
x = df[['movie_id']].copy()
x=scalar.fit_transform(x)
from sklearn.cluster import KMeans
model_labels={}
i_score=[]
s_score=[]
centroids={}
for k in range (2,12):
    #model builing
    model=KMeans(n_clusters=k, random_state=42, n_init=10)
    #model fit
    model=model.fit(x)
    labels=model.labels_
    model_labels[k]=labels
    # Inertia
    i_score.append(model.inertia_)
    # Silhouette score
    s_score.append(silhouette_score(x, labels))
    # Centroids
    centroids[k] = model.cluster_centers_
    print(f" --- inertia{model.inertia_}silhouette{silhouette_score(x,labels)}")


#PLOT:

import matplotlib.pyplot as plt
fig , ax=plt.subplots(2,1,sharex=True)
plt.subplot(211)
plt.plot(range(2,12),i_score,"o--")
plt.title('Inertia')
plt.subplot(212)
plt.plot(range(2,12),s_score,"o--")
plt.title('Silhouette Score')
plt.ylabel('silhouette')
plt.xlabel("k")
k_label=model_labels[3]
k_labels = k_label
k_centroid=centroids[3]

fig,ax=plt.subplots(figsize=(5,5))
plt.scatter(x[k_labels==0,0],x[k_labels==0,1],c="green",label="cluster1")
plt.scatter(x[k_labels==1,0],x[k_labels==1,1],c="yellow",label="cluster
2")
plt.scatter(x[k_labels==2,0],x[k_labels==2,1],c="blue",label="cluster3")
plt.scatter(x[k_labels==3,0],x[k_labels==3,1],c="red",label="cluster4")
plt.scatter(k_centroid[:,0],k_centroid[:,1],c="red", marker='X', s=200,
label='Centroids')
plt.xlabel("Standardized movie_id")
plt.ylabel("Jitter (for visualization only)")
plt.legend()
plt.show()
```

OUTPUT:





RESULT:

Thus , the implementation of K Means Clustering has been successfully executed and the output was verified.

EX NO:6B	HIERARCHICAL CLUSTERING
DATE:	

AIM :

To understand and implement Hierarchical Clustering.

PROCEDURE :

- Collect and preprocess the dataset by selecting the relevant input features.
- Compute the distance (similarity) between all data points.
- Treat each data point as an individual cluster initially.
- Iteratively merge the two closest clusters based on a linkage method (single, complete, or average).
- Continue merging clusters until all data points form a single cluster or the desired number of clusters is reached.
- Represent the clustering process using a dendrogram.
- Analyze the dendrogram to determine the optimal number of clusters.

CODE :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
d=pd.read_csv("/content/drive/MyDrive/717824I363/datasets/moive recommendation.csv")
d.head()
```

OUTPUT:

	movie_id	title	cast	crew	Unnamed: 4	Unnamed: 5	Unnamed: 6	Unnamed: 7	Unnamed: 8	Unnamed: 9	...	Unnamed: 1255	Unnamed: 1256
count	4812	4813	4813	4809	10	10	9	9	9	9	...	1	1
unique	4812	4808	4771	4782	10	7	9	7	9	9	...	1	1
top	25975	""gender"": 0	[]	[]	""name"": ""Ronn Wong""}	""gender"": 0	""department"": ""Costume & Make-Up""	""gender"": 0	""id"": 1463383	""job"": ""Set Dressing Artist""	...	""gender"": 0	""id"": 1645398 ""
freq	1	3	43	28	1	4	1	3	1	1	...	1	1

 # Preprocessing

```
d=d.iloc[1:]
d.isnull().sum()
```

OUTPUT:

	0
title	0
cast	0
crew	4
Unnamed: 4	4803
Unnamed: 5	4803
...	...
Unnamed: 1260	4812
Unnamed: 1261	4812
Unnamed: 1262	4812
Unnamed: 1263	4812
Unnamed: 1264	4812

1264 rows × 1 columns

 # Encoding

```
from sklearn import preprocessing
label_encoder=preprocessing.LabelEncoder()
d['Gender']=label_encoder.fit_transform(d['Gen
der'])
```

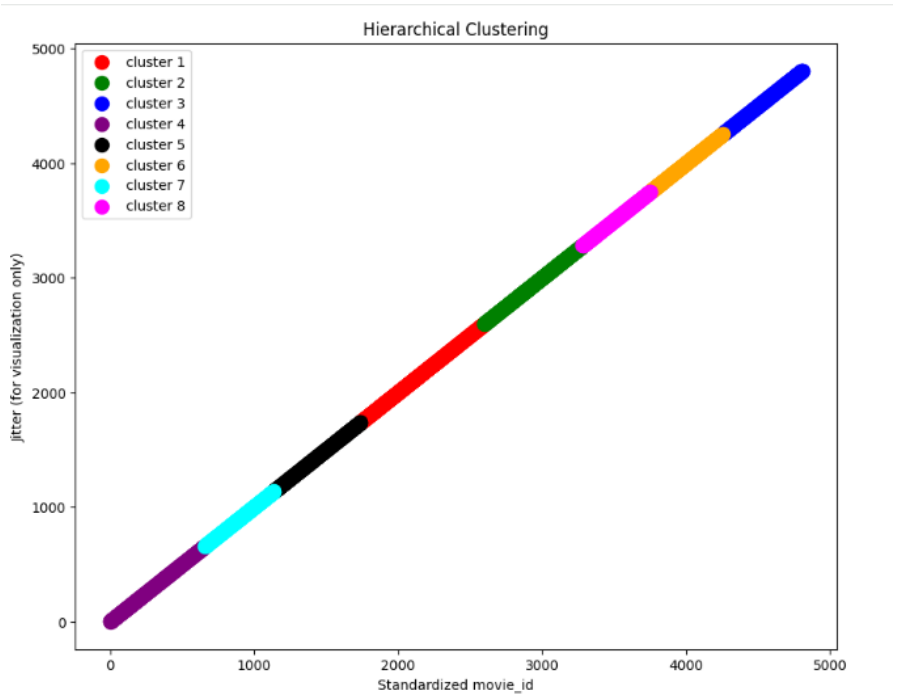
#Model

```
from sklearn.cluster import AgglomerativeClustering
hc=AgglomerativeClustering(n_clusters=8)
x = d.iloc[:, [3, 4]].values
y_hc=hc.fit_predict(x)
```

#Show the hierarchical clustering

```
import matplotlib.pyplot as plt
plt.scatter(x[y_hc==0,0],x[y_hc==0,1],s=100,c='red',label='cluster1')
plt.scatter(x[y_hc==1,0],x[y_hc==1,1],s=100,c='green' ,label='cluster2')
plt.scatter(x[y_hc==2,0],x[y_hc==2,1],s=100,c='yellow',label='cluster3')
plt.scatter(x[y_hc==3,0],x[y_hc==3,1],s=100,c='orange',label='cluster4')
plt.scatter(x[y_hc==4,0],x[y_hc==4,1],s=100,c='violet',label='cluster5')
plt.title('hierarchical clustering model')
plt.xlabel("Standardized movie_id")
plt.ylabel("Jitter (for visualization only)")
plt.show()
```

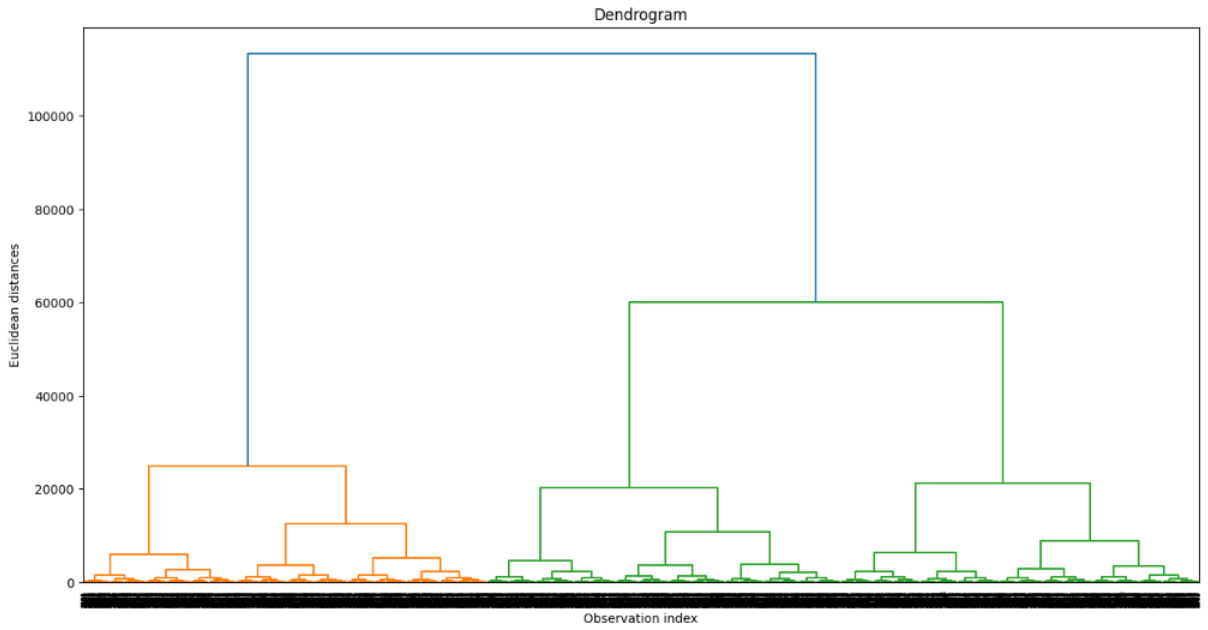
OUTPUT:



 # Dendrogram

```
import matplotlib.pyplot as plt
import scipy.cluster.hierarchy as sch
plt.figure(1,figsize=(16,8))
dendrogram=sch.dendrogram(sch.linkage(clustering_data,method="ward"))
plt.title("Dendrogram")
plt.xlabel("Observation index")
plt.ylabel("Euclidean distances")
plt.show()
```

OUTPUT:



PREPARATION	**30**	
LAB PERFORMANCE	**30**	
REPORT	**40**	
TOTAL	**100**	
INITIAL OF THE FACULTY		

RESULT:

Thus , the implementation of Hierarchical Clustering has been successfully executed and the output was verified.